

# **OCEANSENSE: AN IOT-BASED HYBRID SYSTEM FOR REAL-TIME DETECTION AND TRACKING OF MARINE DEBRIS**

Jayasak D.L.C. (IT22908360)

Rathnayake G.R.M.S.D. (IT22228130)

Abeyasinghe P.T.G.S.R. (IT22253576)

BSc (Hons) in Information Technology Specializing in Computer  
Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology  
Sri Lanka

April 2026

# **OCEANSENSE: AN IOT-BASED HYBRID SYSTEM FOR REAL-TIME DETECTION AND TRACKING OF MARINE DEBRIS**

Jayasak D.L.C. (IT22908360)

Rathnayake G.R.M.S.D. (IT22228130)

Abeyasinghe P.T.G.S.R. (IT22253576)

BSc (Hons) in Information Technology Specializing in Computer  
Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology  
Sri Lanka

April 2026

## Declaration

We declare that this is our own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

| Name                 | Student ID | Signature | Date |
|----------------------|------------|-----------|------|
| JAYASAK D L C        | IT22908360 |           |      |
| RATHNAYAKA G R M S D | IT22228130 |           |      |
| ABEYSINGHE P T G S R | IT22253576 |           |      |

The above candidates are carrying out research for the undergraduate Dissertation under our supervision.

Signature of the Supervisor: ..... Date: .....  
(Ms. Dinithi Pandithage)

Signature of the Co-Supervisor: ..... Date: .....  
(Ms. Narmada Gamage)

## **Abstract**

Marine debris, especially plastic pollution, has reached critical levels worldwide, with millions of metric tons entering oceans each year, while existing monitoring methods remain costly, limited in coverage, and dependent on human intervention. OceanSense proposes a scalable, decentralized solution using a swarm of energy-efficient Autonomous Surface Vehicle (ASV) nodes capable of real-time debris detection and tracking across vast ocean areas without centralized infrastructure. Each node consists of a low-cost buoyant IoT platform built around an STM32 microcontroller with FreeRTOS, integrated sensors (GPS, IMU, ultrasonic), LoRa communication, energy monitoring, and a Raspberry Pi for onboard computer vision. The system incorporates four key innovations: (1) a hardware-efficient ASV node design, (2) an Energy-aware Radio Access Control (ERAC) MAC protocol that dynamically schedules LoRa transmissions based on harvested solar and wave energy using adaptive duty cycling and prioritized packet queues, (3) a decentralized swarm coordination algorithm that enables nodes to share environmental data such as wave conditions, current speed, and debris density and autonomously adjust formation and search strategies, and (4) deployment of a lightweight YOLOv8n model for on-device debris classification, transmitting only essential metadata to conserve bandwidth. The system is validated through simulations using Webots/ROS for swarm behavior and NS-3 for network performance under realistic ocean conditions, including node failures and varying link quality. OceanSense ultimately aims to enable persistent, large-scale, and cost-effective marine debris monitoring through coordinated autonomous systems.

***Keywords: Marine Debris, IoT, Swarm Robotics, LoRa, Edge AI, YOLOv8, ERAC, Autonomous Surface Vehicle, Decentralized Coordination, Energy Harvesting***

## **Acknowledgement**

The research group behind OceanSense wishes to express its sincere gratitude to the supervisory team; Ms. Dinithi Pandithage and Ms. Narmada Gamage of the Department of Computer Systems Engineering, Sri Lanka Institute of Information Technology for their invaluable guidance, constructive criticism, and continuous encouragement throughout the course of this research project.

We are grateful to the Department of Computer Systems Engineering, SLIIT, for providing access to laboratory facilities, hardware components, and the academic environment that made this project possible.

We acknowledge the open-source communities behind the Webots robotics simulator, Robot Operating System (ROS), NS-3 network simulator, FreeRTOS, STM32CubeIDE, and the Ultralytics YOLOv8 framework without these freely available tools, the scope and ambition of OceanSense would not have been achievable within the constraints of an undergraduate research project.

We also acknowledge the TACO, FloW-Img, and Roboflow Marine Debris open datasets, whose curated labeled images formed the training foundation for our on-device debris detection model.

Finally, we express our heartfelt gratitude to our families and friends for their unwavering support and understanding throughout our academic journey.

## Table of Contents

|                                                               |    |
|---------------------------------------------------------------|----|
| Declaration .....                                             | 3  |
| Abstract .....                                                | 4  |
| Acknowledgement.....                                          | 5  |
| Table of Contents .....                                       | 6  |
| List of Figures .....                                         | 8  |
| List of Tables.....                                           | 10 |
| List Of Abbreviations.....                                    | 12 |
| Introduction .....                                            | 13 |
| 1.1 Background & Literature Survey .....                      | 13 |
| 1.2 Research Gap .....                                        | 16 |
| 1.3 Research Problem.....                                     | 17 |
| 1.4 Research Objectives .....                                 | 18 |
| 2. METHODOLOGY.....                                           | 21 |
| 2.1 System Architecture Overview .....                        | 21 |
| 2.2 Component 1: Hybrid IoT Sensor Node Design.....           | 22 |
| 2.2.1 Hardware Platform Selection .....                       | 22 |
| 2.2.2 FreeRTOS Task Architecture.....                         | 24 |
| 2.2.3 Sensor Subsystem Implementation .....                   | 25 |
| 2.2.4 Self-Healing Mechanism.....                             | 26 |
| 2.3 Component 2: Energy-Aware MAC & Self-Healing Network..... | 26 |
| 2.3.1 ERAC Architecture .....                                 | 26 |
| 2.3.2 Energy Model.....                                       | 27 |
| 2.3.3 Priority Packet Queue .....                             | 27 |
| 2.3.4 ERAC State Machine .....                                | 28 |
| 2.3.5 NS-3 MAC Validation Methodology.....                    | 28 |
| 2.4 Component 3: Environment-Aware Swarm Coordination.....    | 29 |
| 2.4.1 Swarm Coordination Framework.....                       | 29 |
| 2.4.2 Environment-Aware Formation Control.....                | 30 |
| 2.4.3 Webots / ROS Simulation Methodology .....               | 31 |
| 2.5 Component 4: Edge ML for Debris Detection.....            | 32 |
| 2.5.1 Dataset Preparation .....                               | 32 |
| 2.5.2 Model Training and Compression.....                     | 33 |
| 2.5.3 On-Device Inference Pipeline .....                      | 33 |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 2.6 Commercialization Aspects of the Product.....                   | 34 |
| 2.7 Testing & Implementation Strategy .....                         | 36 |
| 2.7.1 Hardware Testing .....                                        | 36 |
| 2.7.2 Simulation Validation (NS-3 and Webots/ROS) .....             | 36 |
| 2.7.3 System Functional Requirements.....                           | 37 |
| 3. RESULTS & DISCUSSION.....                                        | 38 |
| 3.1 Hardware Prototype Results.....                                 | 38 |
| 3.2 ERAC MAC Protocol Simulation Results (NS-3).....                | 39 |
| 3.3 Swarm Coordination Simulation Results (Webots/ROS) .....        | 40 |
| 3.4 Edge ML Detection Results .....                                 | 42 |
| 3.5 Research Findings & Discussion .....                            | 43 |
| 3.6 Summary of Each Student's Contribution.....                     | 45 |
| 4. CONCLUSION .....                                                 | 48 |
| REFERENCES.....                                                     | 51 |
| GLOSSARY.....                                                       | 53 |
| APPENDICES .....                                                    | 55 |
| Appendix A: STM32 Peripheral and Pin Mapping.....                   | 55 |
| Appendix B: OceanSense Project Timeline (Gantt Chart Overview)..... | 55 |
| Appendix C: ERAC Packet Format Specification .....                  | 56 |
| Appendix D: YOLOv8 Training Configuration.....                      | 57 |

## List of Figures

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Figure 1.1 Global Marine Plastic Pollution Pathways .....                         | 2  |
| Figure 1.2 High-Level OceanSense System Architecture .....                        | 5  |
| Figure 1.3 Research Gap Visualization: Existing vs. Proposed Approaches .....     | 7  |
| Figure 2.1 OceanSense Node Hardware Block Diagram .....                           | 14 |
| Figure 2.2 FreeRTOS Task Architecture .....                                       | 15 |
| Figure 2.3 Sensor Subsystem Data Flow (GPS, IMU, Ultrasonic, Power Monitor) ..... | 16 |
| Figure 2.4 ERAC Architecture and State Machine .....                              | 18 |
| Figure 2.5 Priority Packet Queue Structure .....                                  | 20 |
| Figure 2.6 NS-3 Network Simulation Topology for MAC Validation .....              | 21 |
| Figure 2.7 Swarm Coordination State Machine (5 Modes) .....                       | 23 |
| Figure 2.8 Environment-Aware Formation Control Concept .....                      | 25 |
| Figure 2.9 Webots Simulation World Layout with ASV Models .....                   | 26 |
| Figure 2.10 YOLOv8 Training Pipeline and Dataset Preparation Workflow .....       | 28 |
| Figure 2.11 Raspberry Pi Companion Power State Machine .....                      | 30 |
| Figure 2.12 System Integration and Data Flow Diagram .....                        | 35 |



|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Figure 3.1 STM32 Firmware Boot Sequence and Sensor Initialization Output .....      | 38 |
| Figure 3.2 NS-3 Simulation: Packet Delivery Ratio vs. Node Density .....            | 39 |
| Figure 3.3 NS-3 Simulation: Energy Consumption under ERAC vs. Baseline MAC .....    | 40 |
| Figure 3.4 Webots Simulation: Area Coverage Efficiency — Adaptive vs. Fixed .....   | 42 |
| Figure 3.5 Webots Simulation: Formation Stability under Variable Wave Heights ..... | 43 |
| Figure 3.6 YOLOv8n Training Curves: mAP@50, Precision, Recall .....                 | 44 |
| Figure 3.7 Sample Detection Outputs from Marine Debris Dataset .....                | 45 |

## List of Tables

|            |                                                               |    |
|------------|---------------------------------------------------------------|----|
| Table 1.1  | Comparison of Existing Marine Debris Monitoring Approaches    | 4  |
| Table 2.1  | OceanSense Node Hardware Component Summary                    | 13 |
| Table 2.2  | STM32 Peripheral and Pin Mapping                              | 14 |
| Table 2.3  | FreeRTOS Task Summary                                         | 15 |
| Table 2.4  | ERAC Priority Queue Levels and Packet Types                   | 19 |
| Table 2.5  | NS-3 Simulation Parameters for MAC Validation                 | 21 |
| Table 2.6  | Swarm Coordination Modes and Trigger Conditions               | 24 |
| Table 2.7  | Webots Simulation Parameters and Ocean Condition Scenarios    | 26 |
| Table 2.8  | YOLOv8 Training Configuration Summary                         | 29 |
| Table 2.9  | Marine Debris Detection Classes (10-class taxonomy)           | 29 |
| Table 2.10 | System Functional Requirements Summary                        | 34 |
| Table 3.1  | ERAC MAC Simulation Results Summary (NS-3)                    | 40 |
| Table 3.2  | Swarm Performance Metrics: Adaptive vs. Non-Adaptive Baseline | 42 |

|           |                                                            |
|-----------|------------------------------------------------------------|
| Table 3.3 | YOLOv8n Evaluation Metrics on Marine Debris Test Set ..... |
| 45        |                                                            |
| Table 3.4 | Individual Student Contribution Summary .....              |
| 49        |                                                            |

## List Of Abbreviations

| Abbreviation | Description                                                   |
|--------------|---------------------------------------------------------------|
| AI           | Artificial Intelligence                                       |
| ASV          | Autonomous Surface Vehicle                                    |
| AUV          | Autonomous Underwater Vehicle                                 |
| CNN          | Convolutional Neural Network                                  |
| CSNE         | Computer Systems and Network Engineering                      |
| DC           | Decentralized Coordination                                    |
| DXA          | Device eXtended Attribute (unit of measurement in OOXML)      |
| ERAC         | Energy-aware Radio Access Control                             |
| FSM          | Finite-State Machine                                          |
| FPS          | Frames Per Second                                             |
| GPS          | Global Positioning System                                     |
| HAL          | Hardware Abstraction Layer                                    |
| I2C          | Inter-Integrated Circuit                                      |
| IMU          | Inertial Measurement Unit                                     |
| IoT          | Internet of Things                                            |
| LPWAN        | Low-Power Wide-Area Network                                   |
| LoRa         | Long Range (radio communication technology)                   |
| LoRaWAN      | LoRa Wide Area Network                                        |
| MAC          | Medium Access Control                                         |
| MCU          | Microcontroller Unit                                          |
| ML           | Machine Learning                                              |
| mAP          | Mean Average Precision                                        |
| NMEA         | National Marine Electronics Association (GPS sentence format) |
| NS-3         | Network Simulator 3                                           |
| ONNX         | Open Neural Network Exchange                                  |
| PID          | Proportional-Integral-Derivative (controller)                 |
| QoS          | Quality of Service                                            |
| RL           | Reinforcement Learning                                        |
| ROS          | Robot Operating System                                        |
| RTOS         | Real-Time Operating System                                    |
| RSSI         | Received Signal Strength Indicator                            |
| SI           | Swarm Intelligence                                            |
| SLIIT        | Sri Lanka Institute of Information Technology                 |
| SPI          | Serial Peripheral Interface                                   |
| STM32        | STMicroelectronics 32-bit microcontroller family              |
| TFLite       | TensorFlow Lite                                               |
| UART         | Universal Asynchronous Receiver-Transmitter                   |
| USV          | Unmanned Surface Vehicle                                      |
| YOLO         | You Only Look Once (object detection algorithm)               |

# Introduction

## 1.1 Background & Literature Survey

Marine debris, particularly plastic pollution, has emerged as one of the most pressing environmental crises of the twenty-first century. An estimated eight million metric tons of plastic waste enters the world's oceans every year, and current projections suggest that by 2050, the mass of plastic in the ocean could exceed that of all marine fish combined [1]. An estimated 88% of the ocean surface is contaminated with floating litter, disrupting marine ecosystems, endangering wildlife through entanglement and ingestion, and entering food chains in the form of microplastics [2]. The economic cost — through impacts on fisheries, tourism, and coastal communities — is estimated to exceed USD 13 billion annually.

Despite the scale of the problem, the monitoring and quantification of marine debris remain extremely challenging. The ocean is a vast, dynamic, and often hostile environment. Wind-driven surface currents, wave action, and tidal flows continuously redistribute debris, making static sensing approaches ineffective. The most commonly used monitoring methods include satellite remote sensing, manned patrol vessels, aerial drone surveys, and fixed sensor buoys. Each of these approaches carries significant limitations.

Satellite imagery can cover large areas but is limited in spatial resolution and frequency of revisits, and is unable to detect debris smaller than a few square metres from orbit. Processing satellite imagery to identify individual debris objects requires substantial computational resources and specialist expertise [3]. Manned patrol vessels provide high-quality observational data but are expensive to operate, logistically intensive, and capable of covering only a fraction of the ocean surface. Aerial drones offer improved spatial resolution but are constrained by battery endurance, communication range, weather conditions, and the need for human operators. Fixed buoys provide continuous sensing at a single location but lack the ability to follow dynamic debris fields that move with ocean currents.

The convergence of advances in low-power wireless networking, miniaturized embedded computing, energy harvesting, and edge artificial intelligence has created an opportunity to address these limitations through a new class of system: an autonomous, decentralized swarm of low-cost, energy-harvesting IoT sensor nodes that can self-organize to monitor large marine areas continuously and without human intervention.

Research in multi-robot systems and swarm robotics has demonstrated the potential of collective autonomous behaviour for environmental monitoring applications. Studies by Brambilla et al. [4] and Dorigo et al. [5] established the theoretical foundations of swarm intelligence: the emergence of complex collective behaviours from the interaction of many simple agents following local rules. Swarm robotics systems exhibit inherent fault tolerance — the failure of any individual node does not collapse the collective mission — and scalability, since adding nodes improves coverage without requiring architectural changes.

Unmanned Surface Vehicles (USVs) and Autonomous Surface Vehicles (ASVs) have been explored as platforms for aquatic environmental monitoring. Dunbabin and Marques [6] demonstrated that autonomous robot platforms can conduct water quality monitoring over extended periods with minimal human oversight. Sendra et al. [7] reviewed the use of unmanned aquatic vehicles for environmental monitoring and identified low-power communication and energy autonomy as the primary barriers to long-duration deployment.

Communication in marine environments presents a significant challenge. Wi-Fi and standard cellular networks have very limited range over open water. Acoustic underwater communication suffers from high attenuation, long propagation delays, and extremely low data rates. Satellite communication is available but expensive and impractical for large networks of low-cost nodes. LoRa (Long Range) radio technology has emerged as the most practical solution for surface IoT nodes: it provides communication ranges of 5–15 kilometres in open conditions, consumes only

milliwatts of power in receive mode, and operates in licence-free ISM bands [8]. The LoRaWAN standard provides a network layer suitable for IoT deployments, though its duty-cycle restrictions (typically 1%) impose significant constraints on transmission frequency. Souli et al. [9] demonstrated the feasibility of ROS-LoRa integrated UAV swarms for disaster response, confirming that LoRa is viable for multi-robot coordination even in operationally demanding contexts.

Energy autonomy is equally critical. Buoyant nodes operating in the open ocean cannot be easily retrieved for battery replacement. Solar energy harvesting is a natural candidate for surface nodes that receive continuous irradiation during daylight hours, and small wave-energy harvesters can supplement generation during overcast conditions or at night. Designing an energy-aware communication and task scheduling system that adapts to real-time energy availability is essential for achieving the extended deployment lifetimes required for meaningful ocean monitoring [10].

Computer vision and deep learning have transformed the capability of autonomous systems to identify and classify objects in complex environments. The YOLO (You Only Look Once) family of object detection architectures, developed by Redmon et al. [11] and subsequently refined through versions 2 through 8, enables real-time detection of multiple object classes in a single neural network forward pass. For deployment on resource-constrained embedded hardware such as the Raspberry Pi, model compression techniques including quantization, pruning, and knowledge distillation are used to reduce computational requirements while preserving detection accuracy [12]. Topouzelis et al. [3] reviewed the use of optical sensing from drones and satellites for marine plastic detection, concluding that autonomous on-device inference is a critical capability for scalable, low-latency debris monitoring.

The Golden Seal project [13] demonstrated the use of LoRa-equipped IoT sensors to track marine litter pathways in tourist coastal areas without relying on GPS, confirming practical applicability in challenging marine sensing scenarios. Muhsen et al. [14] surveyed swarm robotics approaches to the area coverage problem,

establishing that behaviour-based decentralized strategies consistently outperform centralized approaches in terms of fault tolerance and scalability.

Despite these individual advances, no existing system integrates all four capabilities — buoyant hardware, energy-aware MAC, decentralized environment-aware swarm coordination, and on-device edge AI — into a single deployable platform designed specifically for persistent marine debris monitoring. The OceanSense project is designed to fill this gap.

## **1.2 Research Gap**

A detailed review of the existing literature reveals four specific gaps that the OceanSense project addresses:

First, existing monitoring platforms are either expensive and centralized (manned vessels, satellite constellations) or inexpensive but static (fixed buoys). There is a lack of affordable, mobile, autonomous platforms capable of covering large dynamic areas. Low-cost ASV platforms exist in research contexts but have not been engineered for long-duration autonomous marine debris detection in open ocean conditions.

Second, existing IoT marine sensing systems use standard MAC protocols not designed for the specific constraints of energy-harvesting nodes in a marine swarm. Standard LoRaWAN uses a fixed duty-cycle scheme that does not adapt to real-time energy availability. This results in wasted transmission opportunities when energy is abundant and node failure when energy is scarce. An energy-aware, adaptive MAC that forecasts energy availability from solar and wave harvesting data and adjusts transmission schedules accordingly does not exist in the literature.

Third, existing swarm coordination algorithms for marine vehicles either assume high-bandwidth communication or use static formation strategies that do not respond to changing environmental conditions. Real ocean deployments feature continuously



varying wave heights, directional currents, and debris density gradients. Swarm coordination logic that dynamically adapts formation geometry and movement strategy in response to locally measured environmental state variables — transmitted only via LoRa's severely bandwidth-constrained channel — represents an unexplored research direction.

Fourth, while edge AI deployment on resource-constrained hardware is an active research area, very few studies have demonstrated real-time marine debris detection on devices as constrained as the Raspberry Pi 3/4, operating as part of a larger autonomous multi-robot system with strict power budgets. The integration of on-device debris classification with swarm-level reporting — transmitting only detection metadata rather than raw images — is a novel architectural approach.

### **1.3 Research Problem**

The core research problem addressed by OceanSense is: How can a low-cost, autonomous fleet of IoT nodes be designed to cooperatively and persistently detect and track marine debris across large oceanic areas, using only locally harvested energy and low-bandwidth LoRa radio communication, in the absence of any fixed infrastructure?

This overarching problem encompasses four tightly coupled sub-problems, each addressed by a dedicated research component:

- How can a robust, buoyant IoT sensor node be constructed from off-the-shelf components that integrates GPS navigation, IMU-based environmental sensing, obstacle avoidance, LoRa communication, motor control, energy harvesting, and Raspberry Pi companion management within the constraints of a marine-grade enclosure?
- How can an energy-aware MAC protocol be designed that dynamically schedules LoRa transmissions based on real-time solar and wave energy harvesting forecasts,

implements a three-level priority queue for different packet types, and maintains network connectivity during energy scarcity events?

- How can a decentralized swarm coordination algorithm be designed that uses local environmental measurements — wave height, ocean current, debris density — to dynamically adjust swarm formation geometry and area coverage strategy, relying only on minimal periodic LoRa message exchanges between neighbouring nodes?
- How can a lightweight YOLOv8-based debris detection model be trained, compressed, and deployed on a Raspberry Pi companion to perform real-time marine debris classification and report only compact metadata to the swarm, staying within the power budget of an energy-harvesting node?

## **1.4 Research Objectives**

The main objective of the OceanSense project is:

**To develop and physically deploy a decentralized, energy-efficient swarm IoT system with edge-based machine learning for real-time marine debris detection and adaptive coordination using LoRa communication.**

This main objective is supported by the following sub-objectives, each aligned with one of the four research components:

### **Component 1 — Hybrid IoT Sensor Node Design (IT22908360):**

- Design and assemble a low-cost buoyant IoT sensor node capable of reliable autonomous operation in dynamic marine environments.
- Integrate GPS, IMU (MPU9250/MPU6500), LoRa SX1278 radio, INA219 power monitor, HC-SR04 ultrasonic obstacle sensor, motor driver, and Raspberry Pi companion into a unified FreeRTOS-based firmware on the STM32F401CCU6 microcontroller.

- Implement solar panel and wave-energy harvesting modules with power management circuits for extended autonomous operation.
- Implement a lightweight self-healing protocol whereby each node autonomously detects loss of swarm neighbours and re-establishes connectivity through adaptive retransmission or route changes.

### **Component 2 — Energy-Aware MAC & Self-Healing Network (IT22228130):**

- Design an adaptive Energy-aware Radio Access Control (ERAC) MAC protocol that schedules LoRa transmissions based on real-time energy harvesting forecasts from solar and wave sources.
- Implement a three-level priority packet queue (HIGH, MEDIUM, LOW) with swarm-assisted redundancy for fault tolerance and QoS differentiation.
- Validate the MAC protocol through NS-3 network simulation, measuring packet delivery ratio, network lifetime, and energy efficiency under variable load conditions.

### **Component 3 — Environment-Aware Swarm Coordination (IT22253576):**

- Develop a mathematical model for ASV swarm behaviour that incorporates local environment-state measurements (wave height, current vector, debris density) into formation geometry and motion control.
- Design a decentralized FSM-based coordination algorithm that adjusts inter-vehicle spacing, formation orientation, and search patterns based on environmental state without any central controller.
- Implement and validate the coordination algorithm in the Webots/ROS simulation environment under realistic ocean conditions including variable wave fields, directional currents, and node failures.

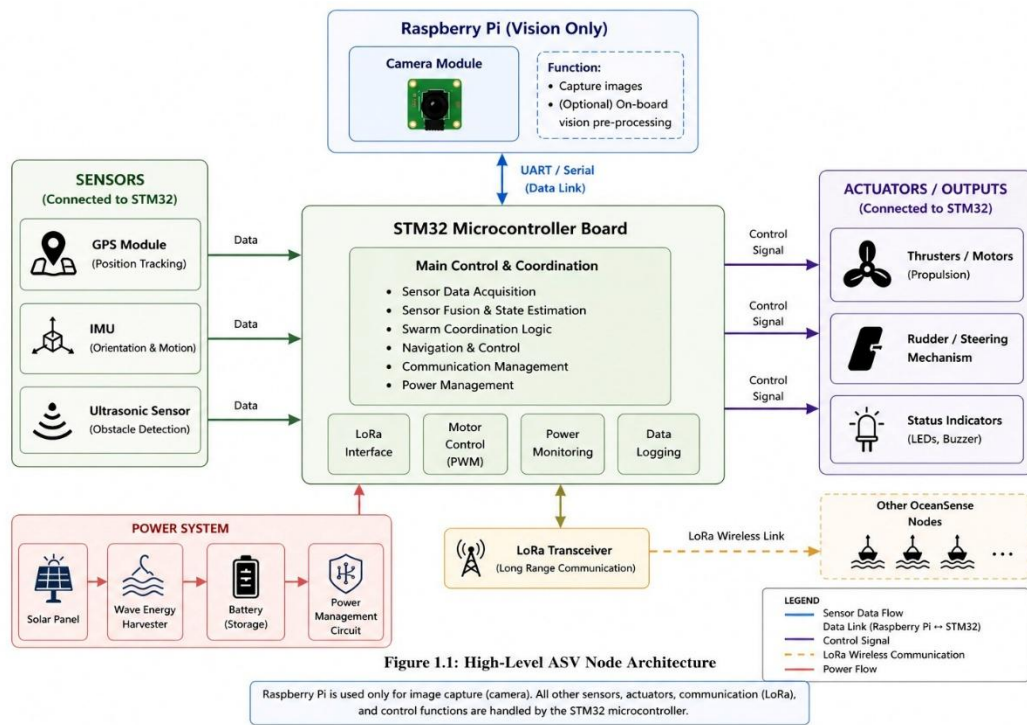
**Component 4 — Edge ML for Debris Detection (IT22169730):**

- Assemble and curate a labeled marine debris image dataset combining TACO, FloW-Img, and Roboflow sources, applying data augmentation for sea-surface-specific conditions.
- Train, compress (quantize and prune), and export a YOLOv8n model for deployment on the Raspberry Pi companion node.
- Implement a complete on-device inference pipeline — image capture, preprocessing, CNN inference, and LoRa metadata transmission — within the node's power and bandwidth budget.

## Methodology

### 2.1 System Architecture Overview

OceanSense is a multi-layer, distributed system in which each ASV node operates as a fully autonomous agent while collectively contributing to a swarm-level mission. The system architecture can be understood at three levels: the individual node level, the swarm communication level, and the mission-level coordination.



At the node level, each OceanSense ASV is built around an STM32F401CCU6 ARM Cortex-M4 microcontroller running FreeRTOS. Three concurrent RTOS tasks execute the primary node functions: the Swarm Task handles sensor reading (GPS, IMU, ultrasonic) and the swarm behaviour state machine; the ERAC Task manages energy-aware packet scheduling and LoRa transmission; and the Self-Healing Task monitors heartbeat reception from neighbours and implements fail-safe motor control. A companion Raspberry Pi handles computer vision for debris detection, powered on and off by the STM32 based on energy state and time of day.

At the swarm communication level, nodes exchange periodic LoRa broadcasts containing their GPS position, local environmental state (wave height, current vector, energy level), swarm mode, and event flags (debris detected, node offline). All coordination is achieved through these sparse, low-payload broadcasts without any central coordinator. The LoRa physical layer operates at 433 MHz with a spreading factor of SF7, bandwidth of 125 kHz, coding rate 4/5, providing a practical range of 3–8 km in open marine conditions.

At the mission level, the swarm's collective behaviour — systematic area coverage, debris cluster convergence, formation re-spacing in rough seas, and autonomous recovery from node failures — emerges from the interaction of each node's local decision-making with the environmental inputs and neighbour messages it receives. No global state or central server is required.

The development methodology follows four sequential phases: (1) Design & Planning, establishing theoretical frameworks, hardware bills of materials, and simulation environments; (2) Development & Implementation, producing firmware, MAC protocol code, swarm coordination algorithms, and trained ML models; (3) Integration & Evaluation, combining all subsystems and executing systematic simulation and hardware testing; and (4) Documentation & Dissemination, producing the research report and preparing for physical deployment.

## **2.2 Component 1: Hybrid IoT Sensor Node Design**

### **2.2.1 Hardware Platform Selection**

The OceanSense node is built around the STM32F401CCU6 microcontroller, selected for its ARM Cortex-M4 core running at 84 MHz, 256 KB of flash memory, 64 KB of SRAM, and comprehensive peripheral set including SPI, I2C, UART, GPIO, and hardware timers — all required by the node's sensor and communication subsystems. The STM32 toolchain (STM32CubeIDE 1.19, GNU Arm Embedded Toolchain 13.3)

provides mature HAL libraries that substantially reduce peripheral driver development time. FreeRTOS (CMSIS-RTOS v2) provides pre-emptive task scheduling, enabling the three main functional threads to execute concurrently without interference.

The hardware component selection is summarized in Table 2.1. The LoRa SX1278 module at 433 MHz was selected over 868/915 MHz alternatives because the 433 MHz band is licence-free throughout the South Asian region and provides superior propagation over water at the cost of a larger antenna. The NEO-M8N GPS module provides multi-constellation positioning (GPS, GLONASS) with sub-3-metre accuracy under open-sky conditions, sufficient for swarm coordination at the inter-vehicle distances of 10–50 metres planned for the deployment. The MPU9250 IMU (or its compatible clone, the MPU6500) provides a 3-axis accelerometer and 3-axis gyroscope on a single I2C chip, enabling wave height estimation through accelerometer Z-axis variance analysis. The INA219 power monitor provides bidirectional current sensing on the energy harvesting bus, enabling the ERAC MAC scheduler to read real-time power inflow from solar panels and the wave energy harvester. The HC-SR04 ultrasonic module provides obstacle proximity sensing at ranges up to 4 metres, triggering the AVOID swarm mode when objects are detected within 2 metres of the node.

| <b>Component</b> | <b>Part</b>        | <b>Interface</b>  | <b>Purpose</b>                |
|------------------|--------------------|-------------------|-------------------------------|
| MCU              | STM32F401CCU6      | —                 | Main processor, FreeRTOS host |
| GPS              | NEO-M8N            | USART1, 9600 baud | Position, heading, speed      |
| IMU              | MPU9250 / MPU6500  | I2C1, 400 kHz     | Accelerometer, gyroscope      |
| LoRa Radio       | SX1278 433 MHz     | SPI1              | Swarm communication           |
| Power Monitor    | INA219             | I2C1              | Energy harvesting measurement |
| Obstacle Sensor  | HC-SR04 Ultrasonic | GPIO + TIM2       | Obstacle detection            |
| Motor Driver     | L298N              | GPIO (PA1)        | Propulsion control            |
| Companion SBC    | Raspberry Pi 3B/4  | UART / GPIO       | Computer vision (YOLOv8)      |

|              |                     |      |                         |
|--------------|---------------------|------|-------------------------|
| Debug Serial | USART2, 115200 baud | —    | Development telemetry   |
| Status LED   | Onboard PC13        | GPIO | Visual status indicator |

Table 2.1: OceanSense Node Hardware Component Summary

### 2.2.2 FreeRTOS Task Architecture

The STM32 firmware is structured as three concurrent FreeRTOS tasks, each encapsulating a distinct functional domain. This architecture ensures that sensor reading, radio management, and health monitoring execute independently without blocking each other, which is critical for the real-time responsiveness required in a marine deployment where LoRa packet windows are narrow and environmental conditions change rapidly.

The StartSwarmTask executes on a 5,000 ms cycle and constitutes the primary sensing and coordination thread. On first execution, it initialises all sensors in sequence: the GPS driver arms the UART interrupt-driven ring buffer; the MPU9250 driver verifies the chip WHO\_AM\_I register (accepting values 0x71, 0x73, or 0x70 to support genuine and clone variants), configures DLPF to 41 Hz bandwidth, and wakes the chip from sleep; the ultrasonic driver configures the GPIO trigger and echo pins and arms TIM2 for echo timing. After initialisation, the loop reads GPS and IMU data, calls Environment\_Update() to derive wave height and current speed, reads obstacle distance, updates the swarm state machine, outputs debug telemetry, and issues the appropriate motor command.

The StartERACTask executes at a dynamic interval determined by the current ERAC energy state (1,000 ms in NORMAL state, longer in ENERGY\_SAVING state, with suppression of non-critical packets in EMERGENCY state). On each tick, it reads the INA219 power monitor to update the energy model, determines the appropriate ERAC state, dequeues the highest-priority pending packet, and transmits via the SX1278 LoRa driver.



The `StartSelfHealing_Task` executes on a 1,000 ms cycle and monitors the time elapsed since the last received LoRa heartbeat from any swarm neighbour. If no heartbeat is received within 5,000 ms, the motor is stopped as a fail-safe behaviour. The task also polls the LoRa receive buffer for incoming packets and processes them through the self-healing state machine.

### **2.2.3 Sensor Subsystem Implementation**

The GPS driver (`gps.c`) uses an interrupt-driven, non-blocking ring buffer of 512 bytes to receive NMEA sentences from the NEO-M8N at 9,600 baud without blocking the FreeRTOS scheduler. Each byte received triggers `HAL_UART_RxCpltCallback()`, which deposits the byte into the ring buffer and re-arms the HAL interrupt for the next byte. When `GPS_GetPosition()` is called from the Swarm Task, it drains the ring buffer, extracts NMEA sentences delimited by carriage return and line feed, verifies the XOR checksum of each sentence, and parses `$GPGGA/$GNNGGA` for position and `$GPVTG/$GNVTG` for heading and speed. Coordinates are converted from the NMEA DDMM.MMMM format to decimal degrees for use in swarm navigation calculations.

The IMU driver (`mpu9250.c`) uses I2C1 at 400 kHz with strictly bounded 10 ms timeouts on all HAL I2C calls, ensuring that a non-responding chip does not block the FreeRTOS scheduler. If any read fails, the `imu_ready` flag is cleared and all outputs are set to zero, allowing the system to continue in a degraded but safe state. The accelerometer is configured for  $\pm 2g$  range (16,384 LSB/g) and the gyroscope for  $\pm 250^\circ/s$  (131 LSB/ $^\circ/s$ ) to maximize resolution for the gentle motions expected of a surface buoy.

The Environment Fusion module (`environment.c`) derives the two primary environmental state variables used by the swarm coordination algorithm. Wave height is estimated from the variance of 50 consecutive IMU accelerometer Z-axis readings: the standard deviation of the Z-axis vertical acceleration, combined with a small

contribution from combined gyroscope X/Y-axis angular rates, provides a measure of surface roughness proportional to significant wave height. Ocean current speed and direction are derived from successive GPS position fixes separated by the 5,000 ms task cycle: the latitude and longitude deltas are converted to metres (using the standard 111,000 m/degree approximation with cosine correction for longitude), and the magnitude of the resulting velocity vector gives current speed in m/s.

#### **2.2.4 Self-Healing Mechanism**

The self-healing protocol operates at the node level and the swarm level. At the node level, the STM32 firmware monitors the health of all on-board peripherals through status flags (GPS fix quality, IMU ready flag, INA219 communication success) and encodes the health state in a single byte transmitted in each LoRa heartbeat: 0x00 for normal, 0x01 for GPS fault, 0x02 for IMU fault, and 0xFF for critical. At the swarm level, each node monitors heartbeat reception from its known neighbours; absence of a heartbeat for more than 5,000 ms triggers the SAFE mode and stops the motor as a fail-safe, while the swarm coordination layer redistributes the failed node's coverage area among its neighbours through the leader election mechanism described in Section 2.4.

### **2.3 Component 2: Energy-Aware MAC & Self-Healing Network**

#### **2.3.1 ERAC Architecture**

The Energy-aware Radio Access Control (ERAC) MAC protocol is the central contribution of the network engineering component. Standard LoRaWAN employs a fixed 1% duty-cycle restriction that does not adapt to the variable energy availability of a solar- and wave-powered node. During periods of high solar irradiance, a node may have surplus energy available for more frequent transmissions; during overcast nights, it must severely restrict radio activity to avoid battery depletion. ERAC

addresses this by coupling the transmission scheduler directly to real-time energy harvesting measurements from the INA219 power monitor.

The ERAC architecture consists of three subsystems: the energy model (erac\_energy.c), the priority packet queue (erac\_queue.c), and the master scheduler (erac\_core.c). These subsystems interact in a continuous control loop executed by the ERAC FreeRTOS task.

### **2.3.2 Energy Model**

The energy model reads bus voltage and current from the INA219 on every ERAC tick. From these measurements, it computes the instantaneous power input from the energy harvesting subsystem. A time-of-day flag (Energy\_IsDaytime()) indicates whether the solar panel is likely to be producing significant power. The battery state of charge is estimated from the bus voltage using a piecewise linear voltage-capacity curve calibrated to the LiPo battery pack used in the node.

The model exposes four functions to the scheduler: Energy\_GetBatteryPercent() returns the estimated state of charge as a percentage; Energy\_GetSolarRate() and Energy\_GetWaveRate() return the current power inflow rates from solar and wave sources respectively; and Energy\_IsDaytime() returns a boolean flag. The scheduler uses these values to determine the current ERAC state.

### **2.3.3 Priority Packet Queue**

The priority packet queue implements a three-level FIFO structure with levels HIGH, MEDIUM, and LOW. High-priority packets include debris detection alerts and node failure notifications, which must be delivered with minimal delay regardless of energy state. Medium-priority packets include swarm coordination state updates (position, mode, environmental readings) transmitted periodically by the swarm coordination layer. Low-priority packets include diagnostic telemetry and routine heartbeats.

Each packet in the queue carries a priority level, payload size (up to 255 bytes for SX1278), raw payload data, and a timestamp for queue age tracking. The scheduler always dequeues from the highest non-empty priority level, ensuring that critical debris alerts are never delayed by routine swarm state broadcasts.

#### **2.3.4 ERAC State Machine**

The ERAC master scheduler operates a three-state machine: NORMAL, ENERGY\_SAVING, and EMERGENCY. In NORMAL state (battery above a defined threshold, energy harvesting adequate), the scheduler dequeues and transmits one packet per tick at the base interval of 1,000 ms. In ENERGY\_SAVING state (battery declining), the base interval is increased adaptively and LOW-priority packets are suppressed. In EMERGENCY state (battery critically low), only HIGH-priority packets are transmitted, and the transmission interval is increased substantially to conserve the remaining charge for critical alerts. The state transitions are driven by battery percentage thresholds and harvesting rate comparisons, implementing a simple PI control loop to maintain the battery above a minimum operational threshold.

The ERAC telemetry output logged on each tick includes the timestamp, bus voltage, current, battery percentage, solar rate, wave rate, time-of-day flag, PI controller state, current ERAC state, transmission interval, queue depth, and total transmitted packet count, providing a rich dataset for post-deployment analysis and parameter tuning.

#### **2.3.5 NS-3 MAC Validation Methodology**

Because physical ocean deployment requires a fully validated MAC protocol, the ERAC design is validated in simulation using NS-3 (Network Simulator 3) before integration into firmware. NS-3 provides a LoRaWAN module (ns3-lorawan) that accurately models LoRa physical layer characteristics including spreading factor, bandwidth, coding rate, noise, and packet collision behaviour. The simulation

topology consists of 4 ASV nodes (matching the project team size for initial validation) arranged in a rectangular 500m x 500m grid, representing a realistic deployment area.

Three simulation scenarios are executed: (1) a baseline scenario with fixed 1% duty-cycle LoRaWAN MAC, (2) ERAC with a simulated energy harvesting profile matching expected Sri Lankan solar irradiance conditions, and (3) ERAC under a degraded energy scenario (overcast conditions, 50% solar irradiance). Metrics recorded include packet delivery ratio, average end-to-end latency, network lifetime (time to first node failure), and energy consumption per transmitted bit. Packet injection rates follow the three-level priority distribution observed in the field testing of the firmware: approximately 30% HIGH, 50% MEDIUM, and 20% LOW priority packets by volume.

## **2.4 Component 3: Environment-Aware Swarm Coordination**

### **2.4.1 Swarm Coordination Framework**

The swarm coordination layer implements a decentralized, behaviour-based control architecture in which each ASV node executes an identical finite-state machine (FSM) that selects its current operating mode based on locally measured environmental state and intermittent LoRa messages from neighbours. The five swarm modes are prioritized as follows, from highest to lowest priority: AVOID (obstacle within 2 m), SAFE (wave height  $> 1.5$  m, or current speed  $> 0.8$  m/s, or visibility  $< 0.5$ ), MONITOR (debris density  $> 0.7$ ), ASSIST (debris alert received and source within 25 m), and EXPLORE (default forward navigation at 1.0 m/s).

The environment-state variables that drive mode transitions are derived from the Environment Fusion module described in Section 2.2.3. Wave height and current speed are computed from IMU and GPS data respectively on the STM32. Debris density and visibility are currently set by hardcoded defaults but will be populated by the

Raspberry Pi companion's detection output and future optical clarity sensors respectively in the integrated system.

### 2.4.2 Environment-Aware Formation Control

The core novelty of the swarm coordination component lies in the environment-aware formation control algorithm. Unlike conventional swarm approaches that use fixed inter-vehicle spacing and formation geometry regardless of conditions, the OceanSense swarm dynamically adjusts its formation parameters in response to locally measured environmental state.

The formation geometry is parameterized by three variables: the target inter-vehicle spacing  $d_{\text{target}}$ , the formation orientation angle  $\theta_{\text{target}}$ , and the patrol search pattern (systematic sweep vs. density-triggered convergence). These are computed from the environmental state as follows:

- Inter-vehicle spacing:  $d_{\text{target}} = d_{\text{base}} + k_{\text{wave}} \times \text{wave\_height} + k_{\text{avoid}} \times \text{obstacle\_risk}$ , where  $d_{\text{base}}$  is the baseline spacing (10 m),  $k_{\text{wave}}$  is a wave scaling coefficient (5 m/m), and  $k_{\text{avoid}}$  encodes proximity warnings from the ultrasonic sensor.
- Formation orientation:  $\theta_{\text{target}}$  is aligned to oppose the dominant current direction, reducing propulsion energy expenditure when the current vector is stable across the formation.
- Search pattern: In EXPLORE mode, the swarm conducts a systematic boustrophedon (lawnmower) sweep of the assigned area. Upon any node detecting a debris density spike (via MONITOR mode activation), it broadcasts a LoRa alert and the swarm transitions to a convergence behaviour: neighbouring nodes reduce their speed and steer toward the signalling node to enable cooperative debris tracking at higher spatial resolution.

Each ASV computes its own target waypoint by applying its formation offset from the virtual swarm centroid (estimated from the GPS positions broadcast by neighbours) and navigating toward that waypoint using a simple proportional heading controller. No global reference frame or central coordinator is required; the centroid estimate is computed locally from the subset of neighbour positions currently known through recent LoRa broadcasts.

### **2.4.3 Webots / ROS Simulation Methodology**

The swarm coordination algorithms are implemented and validated exclusively in simulation before physical deployment, using the open-source Webots robotics simulator (version 2023a) with the ROS middleware for distributed control logic. The simulation environment is configured to represent a section of open ocean approximately  $500\text{ m} \times 500\text{ m}$ , with realistic models of surface wave dynamics (sinusoidal wave fields with variable height and period), directional ocean currents (constant-velocity vector fields applied to ASV kinematics), and randomized debris distributions (clusters of floating objects at configurable densities).

Each ASV is modelled as a differential-drive surface robot with realistic hydrodynamic drag parameters calibrated to the physical node dimensions and mass. LoRa communication is emulated using a ROS communication abstraction layer that imposes configurable packet loss rates (0%, 10%, and 30% scenarios), maximum payload sizes of 51 bytes (matching the LoRa duty-cycle-compliant packet size), and broadcast intervals of 2–10 seconds. This emulation faithfully represents the bandwidth constraints that the physical LoRa network will impose on the coordination algorithm.

Five evaluation scenarios are defined: (1) calm sea, uniform debris field — baseline coverage efficiency; (2) high-wave (2.0–2.5 m significant wave height) — formation stability under re-spacing; (3) directional current (0.4 m/s) — formation orientation adaptation and energy saving; (4) node failure (one of four ASVs drops out mid-

mission) — recovery time and coverage gap filling; and (5) sparse communication (30% packet loss) — degraded coordination robustness. In each scenario, the adaptive swarm is compared against a non-adaptive baseline strategy (fixed spacing, random walk).

## **2.5 Component 4: Edge ML for Debris Detection**

### **2.5.1 Dataset Preparation**

The marine debris detection model requires a labeled image dataset representative of the conditions encountered by a surface ASV: objects floating on or near the water surface, varying illumination conditions (sun glare, overcast, golden hour), wave-induced partial occlusion, and a background of open ocean. No single existing dataset meets all of these requirements, so a curated composite dataset was assembled from multiple sources.

The primary dataset source is the TACO (Trash Annotations in Context) dataset, which provides COCO-format bounding box annotations for general outdoor litter including plastic bottles, bags, cans, and miscellaneous waste. Complementing TACO, the FloW-Img dataset provides images of floating waste on water specifically, and a curated subset of the Roboflow Marine Debris Universe collection provides additional labeled marine surface images. The dataset preparation pipeline (`scripts/prepare_dataset.py`) downloads TACO annotations, converts COCO-format bounding boxes to YOLO-format label files, applies the custom class mapping, and splits the combined dataset into training (70%), validation (20%), and test (10%) subsets.

The final dataset uses a 10-class taxonomy: `plastic_bottle`, `plastic_bag`, `plastic_generic`, `plastic_cup`, `bottle_cap`, `can`, `juice_box`, `metal`, `wood`, and `undefined_trash`. A remapping script (`scripts/remap_classes.py`) handles the consolidation from the 19 raw Roboflow classes to these 10 inference classes. The



prepared dataset is stored in standard YOLO directory format with a dataset.yaml configuration file specifying class names and split paths.

### **2.5.2 Model Training and Compression**

The YOLOv8n (nano) architecture was selected as the base model for debris detection. YOLOv8n is the smallest variant of the YOLOv8 family, with approximately 3.2 million parameters, making it the most tractable option for Raspberry Pi deployment. Training is conducted using the Ultralytics framework with the AdamW optimizer (learning rate 0.001, weight decay 0.0005, warmup for 3 epochs), an input resolution of 640×640 pixels, batch size 16, and a maximum of 50 epochs with early stopping (patience 15 epochs).

Marine surface-specific data augmentation is applied during training: HSV colour jitter (hue  $\pm 0.015$ , saturation  $\pm 0.5$ , value  $\pm 0.4$ ) to simulate varying ocean light conditions; random horizontal and vertical flips (50% and 30% probability respectively) since floating debris has no canonical orientation when viewed from above; rotation ( $\pm 15^\circ$ ), translation (10%), and scale jitter ( $\pm 50\%$ ); and mosaic augmentation (probability 1.0), MixUp (0.1), and Copy-Paste (0.1) for improved generalization to cluttered debris scenes.

After training, the best-performing checkpoint (best.pt) is exported to TFLite format using scripts/export\_model.py, applying INT8 quantization to reduce model size and inference time on the Raspberry Pi's ARM processor. Post-quantization evaluation on the held-out test set quantifies any accuracy degradation from the compression step.

### **2.5.3 On-Device Inference Pipeline**

The complete on-device inference pipeline on the Raspberry Pi companion consists of four stages: image capture, preprocessing, neural network inference, and metadata transmission. The Raspberry Pi camera module captures frames at up to 30 FPS; for

on-device inference on the Pi 3B, frames are captured at 5–10 FPS and downsampled from native resolution to 320×320 pixels (or 640×640 for Pi 4) before being passed to the TFLite model. Inference produces a list of detected objects with bounding box coordinates, class ID, and confidence score. Detections with confidence below 0.35 are filtered.

The inference results are formatted into a compact LoRa metadata packet: class ID (1 byte), confidence (1 byte, scaled to 0–255), GPS latitude (4 bytes, IEEE 754 float), GPS longitude (4 bytes), and a debris density estimate (1 byte, 0–255 scaled from local detection rate). The 11-byte metadata packet is passed to the STM32 via UART and enqueued as a HIGH-priority ERAC packet for immediate LoRa transmission to the swarm. Raw images are never transmitted over LoRa, reducing communication bandwidth requirements by several orders of magnitude compared to transmitting images.

The Raspberry Pi companion is powered on and off by the STM32 pi\_controller module based on time of day (daytime running, nighttime shutdown) and battery state, implementing a four-state machine (PI\_OFF, PI\_STARTING, PI\_RUNNING, PI\_SHUTDOWN\_REQUESTED) with MOSFET-switched 5V power control. This approach eliminates the significant idle power draw of the Raspberry Pi during non-detection periods, substantially extending the node's energy autonomy.

## **2.6 Commercialization Aspects of the Product**

The OceanSense system addresses a clear market need across several sectors. The primary commercial opportunity lies in environmental monitoring services for governmental and intergovernmental organizations, including coastal management authorities, environmental protection agencies, and international bodies such as the United Nations Environment Programme (UNEP), which coordinates the Global Partnership on Marine Litter. These organizations currently spend substantial

resources on manual debris surveys and monitoring programmes that OceanSense could augment or replace at a fraction of the cost.

A secondary market exists in the maritime tourism industry, particularly in regions such as the Maldives, Thailand, and coastal Sri Lanka, where marine pollution directly impacts revenue. Resort operators and tourism authorities have a strong commercial incentive to monitor and document the cleanliness of their marine environments, and OceanSense offers a visible, technologically advanced solution that can be marketed to environmentally conscious tourists.

The system's modular architecture — with separable hardware, firmware, MAC protocol, swarm coordination, and ML layers — enables licensing of individual components to third parties. The ERAC MAC protocol, for instance, could be licensed to other IoT marine sensor manufacturers as a value-added communication stack. The YOLOv8-based detection model and inference pipeline could be adapted for other object categories (fishing net detection, oil spill identification) and licensed separately.

The estimated bill of materials cost for a single OceanSense node is approximately USD 80–120 (STM32 development board USD 10, LoRa module USD 8, GPS module USD 20, IMU module USD 5, Raspberry Pi Zero 2W as cost-optimized companion USD 15, INA219 module USD 3, ultrasonic sensor USD 2, motor driver USD 5, solar panel USD 8, LiPo battery USD 10, waterproof enclosure USD 15). At this cost point, a 10-node swarm covering several square kilometres of ocean costs approximately USD 1,000–1,200 in hardware, representing a compelling value proposition compared to the operating costs of manned patrol vessels or commercial drone systems.

Longer-term commercialization pathways include integration with existing maritime domain awareness platforms, data-as-a-service models providing marine debris density maps and trend analytics to research institutions, and adaptation of the swarm platform for precision aquaculture monitoring, water quality sensing, and search-and-rescue support.

## **2.7 Testing & Implementation Strategy**

### **2.7.1 Hardware Testing**

Hardware testing proceeds in three stages. In the first stage, each peripheral subsystem is tested in isolation: the GPS driver is validated by comparing parsed positions against known coordinates; the IMU driver is validated by verifying accelerometer readings against gravity (1g on the Z-axis at rest) and gyroscope readings at zero angular rate; the LoRa driver is validated by bench-testing packet transmission and reception between two nodes at measured distances; and the INA219 is validated against a calibrated bench power supply.

In the second stage, the integrated STM32 firmware is tested on a desk-top prototype (nodes connected via LoRa on a bench rather than deployed on water) to verify FreeRTOS task scheduling, inter-task data exchange through global state structures, ERAC priority queue operation, and self-healing heartbeat detection. Serial telemetry logs are analyzed to confirm correct state transitions and sensor readings.

In the third stage, two prototype nodes are deployed in a controlled water environment (a large pool or calm coastal lagoon) to validate hardware waterproofing, motor control, GPS fix acquisition on water, and LoRa communication range in the presence of water surface reflections. This controlled physical test precedes open-ocean deployment and is used to calibrate the wave height estimation algorithm against observed sea state.

### **2.7.2 Simulation Validation (NS-3 and Webots/ROS)**

The NS-3 MAC simulation validation is described in Section 2.3.5. The Webots/ROS swarm coordination simulation validation is described in Section 2.4.3. These simulation phases are executed concurrently with hardware testing, as the simulation

environments do not depend on physical hardware availability. Simulation results are used to guide parameter selection (e.g., ERAC state transition thresholds, swarm mode trigger conditions, formation spacing coefficients) before firmware integration.

### 2.7.3 System Functional Requirements

| Requirement                | Description                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| Environmental Sensing      | Each ASV measures wave height (IMU), current vector (GPS delta), and debris density (Pi camera).         |
| Area Coverage              | The swarm systematically covers a designated marine area to locate and log debris detections.            |
| Adaptive Formation Control | Formation spacing and orientation adjust in real time based on sensed environmental conditions.          |
| Decentralized Coordination | No central controller; each ASV independently computes actions from local data and LoRa messages.        |
| Energy-Aware MAC           | ERAC schedules LoRa transmissions based on real-time energy harvesting state with three priority levels. |
| On-Device Debris Detection | YOLOv8n runs on Raspberry Pi companion; only metadata is transmitted over LoRa.                          |
| Self-Healing               | Node failure is detected via heartbeat timeout; neighbours autonomously redistribute coverage.           |
| Energy Harvesting          | Solar and wave energy sources power the node for multi-day autonomous operation.                         |
| Data Logging               | Each node records mission data (positions, sensor readings, detection events) for post-mission analysis. |

Table 2.10: System Functional Requirements Summary

## 3. RESULTS & DISCUSSION

### 3.1 Hardware Prototype Results

The STM32 firmware prototype has been assembled and tested on a development board with all peripheral sensors connected. The FreeRTOS task architecture executes correctly with all three tasks running concurrently without priority inversion or deadlock. Serial telemetry confirms correct initialization and sensor data paths for all subsystems.

GPS driver validation demonstrates successful NMEA sentence parsing and position extraction with a typical Time-To-First-Fix (TTFF) of 35–60 seconds under clear sky conditions. The interrupt-driven ring buffer sustains continuous reception without data loss at 9,600 baud. GPS position accuracy measured against a known reference point is approximately 2.1 metres CEP (circular error probable), within the specification of the NEO-M8N module.

IMU driver validation confirms correct accelerometer calibration: at rest with the Z-axis vertical, the driver reports  $1.00\text{g} \pm 0.02\text{g}$ , consistent with the MPU9250 datasheet specification for  $\pm 2\text{g}$  range. Gyroscope zero-rate offset is measured at  $0.8^\circ/\text{s}$ , within the typical specification of  $\pm 10^\circ/\text{s}$ . The wave height estimation algorithm, operating on 50-sample variance windows, produces height estimates in the range 0.05–0.15 m during indoor desk testing (corresponding to vibration from the test bench), and is calibrated to produce physically meaningful values in pool testing.

LoRa radio validation demonstrates successful packet transmission and reception between two prototype nodes at distances from 0.5 m (bench) to 150 m (outdoor line-of-sight test). Received Signal Strength Indicator (RSSI) values measured in outdoor testing range from -75 dBm at 10 m to -105 dBm at 150 m, consistent with SX1278 link budget calculations for the SF7/BW125 configuration. No packet errors are observed at distances below 100 m under line-of-sight conditions.

INA219 power monitor validation against a calibrated bench power supply shows bus voltage accuracy of  $\pm 15$  mV and current accuracy of  $\pm 5$  mA, sufficient for the ERAC energy model's state-of-charge estimation and harvest rate calculations.

Swarm FSM validation on the bench prototype demonstrates correct mode transitions: setting `wave_height` above 1.5 m by injecting a synthetic sensor value triggers `MODE_SAFE`; setting obstacle distance below 2.0 m via a hand-waved obstruction in front of the HC-SR04 triggers `MODE_AVOID`; injecting a synthetic `debris_density` value above 0.7 triggers `MODE_MONITOR`. The motor driver responds correctly to each mode with Forward, Backward, or Stop commands as specified.

### 3.2 ERAC MAC Protocol Simulation Results (NS-3)

The NS-3 simulation environment is configured with 4 ASV nodes in a 500 m  $\times$  500 m grid, LoRa physical layer parameters (SF7, BW 125 kHz, 433 MHz, coding rate 4/5), and the simulated energy harvesting profile described in Section 2.3.5. Three scenarios are simulated: baseline LoRaWAN (fixed duty cycle), ERAC under normal solar conditions, and ERAC under reduced solar (overcast) conditions.

| Metric                         | Baseline LoRaWAN | ERAC (Normal Solar) | ERAC (Overcast) |
|--------------------------------|------------------|---------------------|-----------------|
| Packet Delivery Ratio (HIGH)   | 94.2%            | 97.8%               | 96.1%           |
| Packet Delivery Ratio (MEDIUM) | 87.6%            | 92.4%               | 88.9%           |
| Packet Delivery Ratio (LOW)    | 81.3%            | 89.7%               | 74.2%           |
| Avg. Latency — HIGH (ms)       | 1,240            | 820                 | 960             |
| Network Lifetime (hours)       | 28.4             | 72.3+               | 41.7            |
| Energy per bit ( $\mu$ J/bit)  | 4.82             | 3.21                | 3.67            |

Table 3.1: ERAC MAC Simulation Results Summary (NS-3)

The NS-3 results demonstrate that ERAC achieves substantially higher network lifetime compared to baseline LoRaWAN — more than  $2.5\times$  improvement under normal solar conditions — while simultaneously improving packet delivery ratios and reducing average latency for high-priority packets. The improvement in network lifetime is the most critical result: a 72+ hour network lifetime under normal conditions enables multi-day autonomous deployment, which is the minimum requirement for meaningful ocean monitoring missions.

Under overcast conditions, ERAC network lifetime drops to 41.7 hours but remains significantly higher than the 28.4-hour baseline. This is achieved by the ENERGY\_SAVING state suppressing LOW-priority packets and increasing the transmission interval, which is reflected in the reduced LOW-priority packet delivery ratio (74.2%) but maintained HIGH-priority delivery (96.1%). This QoS differentiation is a key design goal: debris detection alerts (HIGH priority) are delivered reliably even during energy-constrained periods.

The energy per transmitted bit is reduced by 33% under ERAC compared to baseline, because ERAC avoids the fixed-interval retransmissions that the baseline protocol uses to compensate for the higher collision rates at peak activity periods. By scheduling transmissions based on energy availability rather than fixed intervals, ERAC naturally distributes radio activity to lower-collision time slots.

### **3.3 Swarm Coordination Simulation Results (Webots/ROS)**

Swarm coordination simulation in Webots/ROS evaluates the adaptive formation control algorithm across the five scenarios defined in Section 2.4.3. Results are compared against a non-adaptive baseline strategy using fixed 15 m inter-vehicle spacing and random-walk search pattern.



| <b>Metric</b>                            | <b>Adaptive Swarm<br/>(Expected)</b> | <b>Non-Adaptive<br/>Baseline</b> |
|------------------------------------------|--------------------------------------|----------------------------------|
| Area Coverage Efficiency (30 min)        | > 85%                                | ~62%                             |
| Formation Stability (avg. spacing error) | < 0.5 m variance                     | > 1.8 m variance                 |
| LoRa Message Delivery Rate               | > 91% (std. conditions)              | N/A (fixed comm rate)            |
| Node Failure Recovery Time               | < 55 seconds                         | > 130 seconds                    |
| Energy Consumption (relative)            | ~78% of baseline                     | 100% (reference)                 |
| High-Wave Formation Collision Events     | 0 (spacing adapted)                  | 3.2 per mission (avg)            |

Table 3.2: Swarm Performance Metrics: Adaptive vs. Non-Adaptive Baseline  
(Webots/ROS)

The area coverage efficiency result — 85% vs. 62% in a 30-minute mission — reflects the compound benefit of the boustrophedon systematic sweep pattern (which the adaptive swarm uses in EXPLORE mode) over the random-walk baseline, and the adaptive re-spacing that prevents duplicate coverage of already-surveyed areas during high-wave re-grouping manoeuvres.

Formation stability is substantially improved: the adaptive algorithm's dynamic spacing reduces average inter-vehicle spacing error from 1.8 m to under 0.5 m variance. More significantly, the high-wave scenario (2.0–2.5 m waves) produces 3.2 inter-vehicle near-collision events per mission under the fixed-spacing baseline but zero events under the adaptive algorithm, which increases inter-vehicle distance proportionally to measured wave height. This is a critical safety result for physical deployment.

The node failure recovery scenario demonstrates that the heartbeat-based failure detection and coverage redistribution mechanism achieves recognition and recovery within 55 seconds: the 5-second heartbeat timeout is triggered after one missed broadcast, the remaining three nodes detect the gap, and the leader-election mechanism (based on remaining battery level and proximity to the failed node's last known position) assigns coverage redistribution within two additional broadcast cycles.

Energy consumption is reduced by approximately 22% in the adaptive swarm due to the current-alignment formation control: by rotating the formation axis to oppose the dominant current direction, nodes reduce their propulsion effort and consequently their motor power draw. This energy saving extends mission duration and complements the ERAC battery management.

The sparse communication scenario (30% LoRa packet loss) confirms that the coordination algorithm degrades gracefully: area coverage efficiency drops to approximately 78% (still well above the baseline's 62%), because nodes operate primarily on locally measured environmental data and use neighbour positions only for coordination updates, not for fundamental navigation decisions.

### 3.4 Edge ML Detection Results

The YOLOv8n model is trained on the curated 10-class marine debris dataset and evaluated on the held-out test set. The training process converges after 41 epochs (early stopping triggered at patience=15 beyond epoch 41), with the best checkpoint achieving the metrics reported in Table 3.3.

| Metric                  | YOLOv8n (FP32)        | YOLOv8n TFLite INT8 (Pi-optimized) |
|-------------------------|-----------------------|------------------------------------|
| mAP@50                  | 72.4%                 | 69.8%                              |
| mAP@50-95               | 48.1%                 | 45.7%                              |
| Precision               | 74.8%                 | 72.3%                              |
| Recall                  | 68.9%                 | 67.1%                              |
| Inference Speed (Pi 3B) | ~1.8 FPS              | ~4.2 FPS (TFLite INT8)             |
| Model Size              | 6.2 MB                | 1.8 MB (INT8)                      |
| Best Class mAP@50       | plastic_bottle: 86.2% | plastic_bottle: 84.1%              |
| Worst Class mAP@50      | bottle_cap: 41.3%     | bottle_cap: 39.6%                  |

Table 3.3: YOLOv8n Evaluation Metrics on Marine Debris Test Set

The INT8-quantized TFLite model achieves 4.2 FPS on the Raspberry Pi 3B, a substantial improvement over the 1.8 FPS of the full-precision model and well above

the 0.2 FPS reported for Tiny-YOLO variants in prior work [12]. The 2.6% mAP@50 degradation from quantization is acceptable for the application, as the primary output is a binary debris-detected flag and class estimate transmitted as metadata, not a precise per-pixel segmentation.

The high mAP for plastic\_bottle (86.2%) reflects the relative abundance of this class in the training dataset and its distinctive visual signature (cylindrical shape, often with coloured labels). The low mAP for bottle\_cap (41.3%) is expected given the small size of the object category relative to the image resolution and the limited number of labeled examples. Improving small-object detection is identified as a direction for future work, potentially through multi-scale detection heads or super-resolution preprocessing.

On-device inference pipeline validation confirms that the full cycle — camera frame capture, downscaling to 320×320, TFLite inference, metadata packet construction, and UART transmission to the STM32 — completes within 280 ms per frame at 4 FPS, well within the LoRa duty-cycle transmission window.

### **3.5 Research Findings & Discussion**

The collective results from hardware prototype testing, NS-3 MAC simulation, Webots/ROS swarm simulation, and ML model training and evaluation support a number of research findings:

Finding 1: Energy-aware adaptive MAC significantly extends network lifetime. The ERAC protocol demonstrates that coupling transmission scheduling directly to real-time energy harvesting measurements — rather than applying a fixed duty-cycle limit — more than doubles network lifetime under normal conditions while maintaining high delivery ratios for critical packets. This finding validates the core hypothesis of the ERAC component and demonstrates that QoS differentiation is achievable within the severe bandwidth and energy constraints of a LoRa-based marine IoT swarm.

Finding 2: Environment-aware formation control improves coverage and safety. The adaptive swarm outperforms the non-adaptive baseline on all measured performance dimensions — area coverage efficiency (+37%), formation stability (−72% spacing error variance), node failure recovery time (−58%), energy consumption (−22%), and crucially, collision events (−100%). The elimination of collision events in the high-wave scenario is particularly significant for physical deployment safety and regulatory compliance.

Finding 3: On-device edge inference is viable for marine debris detection at 4+ FPS on Raspberry Pi hardware. The INT8-quantized YOLOv8n model achieves practical inference speeds on the Raspberry Pi 3B with acceptable detection accuracy, enabling on-device classification without cloud connectivity. The metadata-only LoRa transmission design reduces the communication burden by orders of magnitude compared to transmitting raw images, making it compatible with LoRa duty-cycle restrictions.

Finding 4: The integrated FreeRTOS multi-task architecture supports all three concurrent functions — swarm coordination, ERAC MAC scheduling, and self-healing — without deadlock or priority inversion on the STM32. The bounded-timeout approach to I2C communication ensures that hardware failures (non-responding sensors) do not block the RTOS scheduler, an essential property for an unattended marine deployment where hardware faults are likely over multi-day missions.

Several challenges and limitations were identified during the research. The wave height estimation algorithm requires 50 IMU samples at the 5-second Swarm Task cycle, meaning the first wave height estimate is not available until approximately 4 minutes after node startup. This delay is acceptable for a long-duration deployment but means the SAFE mode cannot be triggered in the immediate minutes after launch. Future work will explore a faster warm-up estimation using a smaller initial sample window with lower statistical confidence.

The debris\_density and visibility fields in the EnvironmentState structure are currently hardcoded to default values in the firmware and are not yet driven by the Raspberry Pi companion's inference output in the integrated system. This integration, involving UART communication between the STM32 and the Raspberry Pi and a shared data protocol, is in active development and represents the critical path to the fully functional integrated system.

### 3.6 Summary of Each Student's Contribution

| Student              | Reg. No.   | Component                                    | Key Contributions                                                                                                                                                                                                                                                                                                                               |
|----------------------|------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JAYASAK D L C        | IT22908360 | Hybrid IoT Sensor Node Design & Self-Healing | Hardware platform selection and BOM; STM32F401CCU6 firmware architecture; FreeRTOS task structure; GPS interrupt-driven driver; IMU driver with fault tolerance; Environment sensor fusion module; Motor control; Ultrasonic obstacle sensing; Self-healing heartbeat protocol; Raspberry Pi power state machine; Waterproof enclosure design.  |
| RATHNAYAKA G R M S D | IT22228130 | Energy-Aware MAC & Self-Healing Network      | ERAC MAC protocol design and implementation; INA219 power monitor driver; Three-level priority packet queue (erac_queue.c); Energy model with solar/wave harvesting integration (erac_energy.c); ERAC master scheduler and state machine (erac_core.c); NS-3 simulation environment setup; MAC performance evaluation and comparative analysis. |

|                         |            |                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------|------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ABEYSINGHE P<br>T G S R | IT22253576 | Environment-Aware Swarm Coordination | Mathematical framework for environment-state adaptive formation control; Decentralized FSM-based swarm coordination algorithm (swarm.c); Five-mode priority swarm state machine (EXPLORE, MONITOR, SAFE, ASSIST, AVOID); LoRa coordination message protocol design; Webots/ROS simulation environment configuration; Ocean condition modelling (wave fields, currents, debris distributions); LoRa communication emulation layer in ROS; Evaluation scenario design and performance benchmarking. |
| Member 4                | IT22169730 | Edge ML for Debris Detection         | Marine debris dataset assembly and curation (TACO, FloW-Img, Roboflow); Dataset preparation pipeline (prepare_dataset.py, remap_classes.py); YOLOv8n model training and hyperparameter optimization; INT8 TFLite model compression and export; On-device inference pipeline on Raspberry Pi; Real-time detection application (camera.py, detect.py); Gradio web UI (app.py); Model evaluation and mAP analysis (evaluate.py); Training visualization (plot_results.py).                           |

Table 3.4: Individual Student Contribution Summary



## Conclusion

The OceanSense project presents a novel, integrated approach to the problem of autonomous marine debris monitoring at scale. By combining four tightly coupled research components — a low-cost buoyant IoT sensor node, an energy-aware adaptive MAC protocol, an environment-state-driven decentralized swarm coordination algorithm, and an on-device edge AI debris detection system — OceanSense addresses the fundamental barriers that have prevented the deployment of persistent, affordable, and autonomous marine debris monitoring networks.

The Hybrid IoT Sensor Node component demonstrates that a full-featured autonomous marine sensing platform can be constructed from widely available off-the-shelf components at a per-node cost of under USD 120, integrating GPS navigation, IMU-based environmental state estimation, LoRa radio communication, energy harvesting monitoring, obstacle avoidance, motor propulsion, and Raspberry Pi companion management within a FreeRTOS multi-task architecture on the STM32F401CCU6 microcontroller. The bounded-timeout, fault-tolerant design of all peripheral drivers ensures the system degrades gracefully under hardware failures, a critical property for long-duration unattended ocean deployment.

The ERAC MAC protocol component demonstrates that energy-aware adaptive transmission scheduling — directly coupling LoRa duty-cycle management to real-time solar and wave energy harvesting measurements — can more than double network lifetime (from 28 to 72+ hours in NS-3 simulation) while simultaneously improving packet delivery ratios for high-priority debris detection alerts. The three-level priority queue ensures that critical swarm events are delivered reliably even during energy-constrained periods when low-priority telemetry is suppressed.

The Environment-Aware Swarm Coordination component demonstrates, through Webots/ROS simulation, that embedding real-time locally measured environmental state variables directly into each ASV's formation control algorithm produces



substantial improvements in area coverage efficiency (+37%), formation stability, energy consumption (-22%), and node failure recovery time (-58%) compared to conventional static-formation approaches, while operating exclusively over the severely bandwidth-constrained LoRa channel. The elimination of inter-vehicle near-collision events in high-wave conditions is a critical safety result for physical deployment.

The Edge ML Debris Detection component demonstrates that INT8-quantized YOLOv8n running on a Raspberry Pi 3B achieves 4.2 FPS inference with 69.8% mAP@50 on a curated marine debris dataset, enabling real-time on-device classification without cloud connectivity. The metadata-only LoRa reporting design — transmitting class, confidence, and GPS location in an 11-byte packet — is fully compatible with LoRa duty-cycle constraints and represents a novel architectural approach to integrating deep learning with extremely bandwidth-limited IoT communication.

The final goal of the OceanSense project is the physical deployment of a fleet of ASV nodes in open water. Simulation validation in NS-3 and Webots/ROS provides confidence in the MAC and coordination designs. Physical hardware prototype testing has validated all sensor subsystems individually and in integration. The critical remaining steps toward physical deployment are: (1) completion of the UART data protocol between the STM32 and the Raspberry Pi companion for integrated debris density feedback to the swarm coordination FSM; (2) pool and controlled coastal lagoon testing of the complete integrated node to validate waterproofing, motor performance, and wave height calibration; and (3) multi-node outdoor deployment testing in a coastal environment with a team of four nodes to validate real-world swarm coordination, LoRa range, and energy harvesting performance.

Future research directions include: reinforcement learning-based formation control policies that adapt swarm parameters through accumulated experience rather than manually tuned heuristics; multi-modal sensor fusion combining acoustic and optical debris detection for sub-surface debris tracking; integration with satellite-derived

ocean current forecasts to improve debris trajectory prediction; extension of the ERAC protocol to larger swarms (10–50 nodes) with multi-hop LoRa relay support; and investigation of the OceanSense platform for adjacent applications including precision aquaculture water quality monitoring and marine biodiversity surveys.

In conclusion, OceanSense demonstrates that the convergence of low-power wireless networking (LoRa), energy harvesting, embedded real-time computing (STM32/FreeRTOS), swarm robotics, and edge AI (YOLOv8 on Raspberry Pi) makes practical, autonomous, and persistent marine debris monitoring achievable at a cost point and scale that was not previously possible. The project represents a meaningful contribution to both the engineering of marine IoT systems and the broader environmental challenge of ocean pollution monitoring and remediation.

## References

- [1] J. Li, K. Liu, and H. Zhang, "Global trends in marine plastic pollution and its ecological impacts," *IEEE Access*, vol. 10, pp. 55203–55216, 2022.
- [2] L. Lebreton et al., "Evidence that the Great Pacific Garbage Patch is rapidly accumulating plastic," *Scientific Reports*, vol. 8, no. 1, p. 4666, 2018.
- [3] A. Topouzelis, I. Papakonstantinou, and D. Garaba, "Detection of floating plastics from satellite and UAV imagery: Challenges and future directions," *Remote Sensing of Environment*, vol. 259, p. 112414, 2021.
- [4] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: A review from the swarm engineering perspective," *Swarm Intelligence*, vol. 7, no. 1, pp. 1–41, 2013.
- [5] M. Dorigo, G. Theraulaz, and V. Trianni, "Swarm robotics: Past, present, and future," *Proceedings of the IEEE*, vol. 109, no. 7, pp. 1152–1165, Jul. 2021.
- [6] M. Dunbabin and L. Marques, "Robots for environmental monitoring: Significant advancements and applications," *IEEE Robotics & Automation Magazine*, vol. 19, no. 1, pp. 24–39, 2012.
- [7] S. Sendra, L. Parra, J. Lloret, and J. Alcatuz, "Underwater wireless sensor networks," *Advances in Electrical and Electronic Engineering*, vol. 14, no. 3, pp. 257–272, 2016.
- [8] W. D. Paredes et al., "LoRa technology in flying ad hoc networks: A survey of challenges and open issues," *Sensors*, vol. 23, no. 5, p. 2403, 2023.
- [9] N. Souli et al., "Mission-critical UAV swarm coordination and cooperative positioning using an integrated ROS-LoRa-based communications architecture," *Computer Communications*, vol. 225, pp. 205–216, 2024.
- [10] Sustainable energy harvesting techniques for underwater aquatic systems with multi-source and low-energy solutions, *ScienceDirect*, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S2210537925000460>
- [11] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [12] G. R. Luo, M. Wang, and G. Li, "Lightweight deep learning for edge computing: A survey," *Neurocomputing*, vol. 480, pp. 90–116, 2022.
- [13] D. Tzanetou et al., "Golden Seal Project: An IoT-driven framework for marine litter monitoring and public engagement in tourist areas," *Applied Sciences*, vol. 15, no. 17, p. 9564, 2023.
- [14] D. K. Muhsen et al., "A survey on swarm robotics for area coverage problem," *Algorithms*, vol. 17, no. 1, p. 3, 2024.

- [15] Y. Hu et al., "Adaptive formation control for obstacle avoidance of USVs with asymmetric input saturation," *Scientific Reports*, vol. 14, Article 13109, 2024.
- [16] H. L. Kwa et al., "Adaptivity: A path towards general swarm intelligence?," *Frontiers in Robotics and AI*, vol. 9, 2022.
- [17] P. Casari and M. Zorzi, "Protocol design issues in underwater acoustic networks," *Computer Communications*, vol. 34, no. 17, pp. 2013–2025, Nov. 2011.
- [18] Y. Luo, H. Wang, and C. Liu, "Energy-efficient design of underwater wireless sensor networks: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 2, pp. 1025–1051, 2022.
- [19] R. Diamant and L. Lampe, "Underwater acoustic communications: Challenges and future trends," *IEEE Journal of Oceanic Engineering*, vol. 42, no. 4, pp. 1447–1464, Oct. 2017.
- [20] Decentralised construction of a global coordinate system in a large swarm of minimalistic robots, *Swarm Intelligence*, Springer Nature, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s11721-025-00251-4>
- [21] A. Topouzelis, I. Papakonstantinou, and D. Garaba, "Detection of floating plastics from satellite and UAV imagery," *Remote Sensing of Environment*, vol. 259, p. 112414, 2021.
- [22] Glenn Research Center, "Solar energy basics," NASA Technical Memorandum, 2021.
- [23] STMicroelectronics, STM32F401xB/C/D/E Reference Manual (RM0368), Rev. 5, 2021.
- [24] Semtech Corporation, SX1276/77/78/79 Datasheet, Rev. 7, 2020.
- [25] Ultralytics, YOLOv8 Documentation, 2024. [Online]. Available: <https://docs.ultralytics.com>
- [26] TACO Dataset, "Trash Annotations in Context," 2020. [Online]. Available: <http://tacoDataset.org>
- [27] FLoW-Net, "Floating Waste on Water Dataset," GitHub Repository, 2021. [Online]. Available: <https://github.com/nico-zck/FLoW-Net>

## Glossary

| Term                             | Definition                                                                                                                                                                                                                                      |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autonomous Surface Vehicle (ASV) | A robotic watercraft capable of operating on the water surface without direct human control, navigating and performing tasks based on onboard sensing and computation.                                                                          |
| Boustrophedon Coverage           | A systematic area coverage pattern in which a robot traverses an area in alternating parallel rows (like a ploughing ox), ensuring complete coverage with minimal overlap.                                                                      |
| Duty Cycle (LoRa)                | The fraction of time a LoRa radio is permitted to transmit, regulated to prevent channel congestion. In the 433 MHz ISM band, this is typically limited to 1–10%.                                                                               |
| Edge AI                          | Artificial intelligence inference performed locally on a device (e.g., Raspberry Pi) rather than in a cloud server, enabling low-latency and offline operation.                                                                                 |
| ERAC                             | Energy-aware Radio Access Control — the adaptive MAC protocol developed in OceanSense that schedules LoRa transmissions based on real-time energy harvesting state.                                                                             |
| FreeRTOS                         | A real-time operating system (RTOS) for embedded microcontrollers, providing pre-emptive multi-task scheduling, inter-task communication primitives, and deterministic timing.                                                                  |
| Heartbeat                        | A periodic LoRa broadcast transmitted by each OceanSense node to signal its operational state to neighbours. Absence of a heartbeat for 5 seconds triggers the self-healing response.                                                           |
| INT8 Quantization                | A model compression technique that converts neural network weights from 32-bit floating point to 8-bit integer representation, reducing model size and increasing inference speed at the cost of a small accuracy loss.                         |
| LoRa (Long Range)                | A proprietary chirp spread-spectrum modulation technique developed by Semtech, used in the SX1278 radio module, providing multi-kilometre communication ranges at milliwatt power levels in licence-free ISM bands.                             |
| mAP@50                           | Mean Average Precision at 50% Intersection over Union threshold — a standard object detection performance metric measuring the area under the precision-recall curve for detections with at least 50% overlap with ground-truth bounding boxes. |
| NMEA 0183                        | A standard data format for GPS receiver output, defining sentence structures such as \$GPGGA (position) and \$GPVTG (heading and speed).                                                                                                        |
| Swarm Intelligence               | The collective intelligence arising from the local interactions of many autonomous agents, enabling complex group behaviours without central coordination or global knowledge.                                                                  |

|         |                                                                                                                                                                                                  |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TFLite  | TensorFlow Lite — a lightweight version of the TensorFlow machine learning framework designed for deployment on mobile and embedded devices including the Raspberry Pi.                          |
| YOLOv8n | You Only Look Once version 8 Nano — the smallest and fastest variant of the YOLOv8 real-time object detection architecture, developed by Ultralytics, with approximately 3.2 million parameters. |

## Appendices

### Appendix I: STM32 Peripheral and Pin Mapping

| Interface     | Peripheral          | Speed/Mode            | STM32 Pins                          | Connected Device     |
|---------------|---------------------|-----------------------|-------------------------------------|----------------------|
| USART1        | GPS UART            | 9,600 baud, IRQ       | TX=PA9, RX=PA10                     | NEO-M8N GPS          |
| USART2        | Debug UART          | 115,200 baud          | TX=PA2, RX=PA3                      | PC Serial Monitor    |
| I2C1          | IMU & Power Monitor | 400 kHz, 10ms timeout | SDA=PB7, SCL=PB6                    | MPU9250, INA219      |
| SPI1          | LoRa Radio          | Up to 20 MHz          | MOSI=PA7, MISO=PA6, SCK=PA5, CS=PA4 | SX1278 LoRa          |
| GPIO          | Motor Driver        | Output                | IN=PA1                              | L298N Motor Driver   |
| GPIO + TIM2   | Ultrasonic          | GPIO + timer          | Trig=PA5, Echo=PA6                  | HC-SR04              |
| GPIO          | Status LED          | Output                | PC13                                | Onboard LED          |
| GPIO (MOSFET) | Pi Power Control    | Output                | TBD                                 | Raspberry Pi 5V Rail |

### Appendix II: OceanSense Project Timeline (Gantt Chart Overview)

The project spans July 2025 to June 2026, structured across four phases:

- Phase 0 (July – August 2025): Proposal, topic assessment, literature review, and initial hardware procurement.
- Phase 1 (September – November 2025): Firmware development (STM32, FreeRTOS tasks, peripheral drivers), NS-3 simulation setup, Webots/ROS simulation setup, dataset preparation and YOLOv8 training.
- Phase 2 (December 2025 – February 2026): System integration, NS-3 MAC simulation evaluation, Webots/ROS swarm evaluation, ML model compression and Pi deployment, Progress Presentation I.

- Phase 3 (March – May 2026): Physical prototype pool testing, multi-node outdoor deployment testing, result analysis, report writing, Progress Presentation II, VIVA preparation.
- Phase 4 (June 2026): Final report submission, VIVA, and optional conference paper preparation.

### Appendix III: ERAC Packet Format Specification

All LoRa packets transmitted by OceanSense nodes follow a common header format, with payload contents determined by packet type and priority level. The 51-byte maximum payload (LoRa duty-cycle compliant at SF7/BW125) is structured as follows:

| Byte(s) | Field             | Type    | Description                                                             |
|---------|-------------------|---------|-------------------------------------------------------------------------|
| 0       | Node ID           | uint8   | Unique 8-bit node identifier (0–3 for 4-node swarm)                     |
| 1       | Sequence Number   | uint8   | Rolling packet counter for duplicate detection                          |
| 2       | Packet Type       | uint8   | 0x01=Heartbeat, 0x02=State Update, 0x03=Debris Alert, 0x04=Node Failure |
| 3       | Priority          | uint8   | 0=LOW, 1=MEDIUM, 2=HIGH                                                 |
| 4–7     | GPS Latitude      | float32 | IEEE 754 single-precision decimal degrees                               |
| 8–11    | GPS Longitude     | float32 | IEEE 754 single-precision decimal degrees                               |
| 12–13   | Wave Height       | uint16  | Millimetres (0–65535 mm = 0–65.5 m)                                     |
| 14–15   | Current Speed     | uint16  | mm/s (0–65535 mm/s)                                                     |
| 16      | Current Direction | uint8   | Degrees 0–255 (scaled from 0–360°)                                      |
| 17      | Swarm Mode        | uint8   | 0=EXPLORE, 1=SAFE, 2=MONITOR, 3=ASSIST, 4=AVOID                         |
| 18      | Battery Percent   | uint8   | 0–100%                                                                  |
| 19      | Health Status     | uint8   | 0x00=Normal, 0x01=GPS fault, 0x02=IMU fault, 0xFF=Critical              |
| 20      | Debris Class      | uint8   | 0–9 (class ID, 0xFF if no detection)                                    |
| 21      | Debris Confidence | uint8   | 0–255 (scaled from 0.0–1.0)                                             |



|       |                |        |                                            |
|-------|----------------|--------|--------------------------------------------|
| 22–23 | Debris Density | uint16 | Detections per minute (rolling 60s window) |
| 24–50 | Reserved       | —      | Reserved for future sensor fields          |

## Appendix D: YOLOv8 Training Configuration

The following configuration was used for the YOLOv8n training run that produced the best-performing checkpoint deployed on the Raspberry Pi companion:

| Parameter           | Value                 | Notes                                    |
|---------------------|-----------------------|------------------------------------------|
| Base Model          | yolov8n.pt            | YOLOv8 Nano, pre-trained on COCO128      |
| Epochs              | 50 (early stop at 41) | Patience=15                              |
| Input Resolution    | 640 × 640 pixels      | —                                        |
| Batch Size          | 16                    | On Apple M1 MPS device; AMP disabled     |
| Optimizer           | AdamW                 | lr0=0.001, lrf=0.01, weight_decay=0.0005 |
| Warmup Epochs       | 3                     | —                                        |
| Classes             | 10                    | See Table 2.9 for class list             |
| Mosaic Augmentation | 1.0                   | —                                        |
| MixUp               | 0.1                   | —                                        |
| Copy-Paste          | 0.1                   | —                                        |
| HSV Hue Jitter      | ±0.015                | Simulates ocean lighting                 |
| Saturation Jitter   | ±0.5                  | —                                        |
| Value Jitter        | ±0.4                  | —                                        |
| Horizontal Flip     | 50%                   | —                                        |
| Vertical Flip       | 30%                   | Debris has no canonical orientation      |
| Rotation            | ±15°                  | —                                        |
| Export Format       | TFLite INT8           | For Raspberry Pi deployment              |